

MCMC Calibrator Interview Preparation Notes

Stamatics Society, IITK Project

Prepared for technical interview defense

June 20, 2026

Contents

1	How To Use This Document	5
2	Project Elevator Pitch	5
2.1	The Problem	5
2.2	The Core Idea	5
2.3	What We Estimate	5
3	What The Project Is Not	6
4	Day-Wise And Subtask-Wise Plan	6
4.1	Day 1: Understand The Trading Problem	6
4.2	Day 2: Data Collection With <code>yfinance</code>	7
4.3	Day 3: Log Prices And Hedge Ratio	7
4.4	Day 4: Stationarity And ADF Test	7
4.5	Day 5: Ornstein-Uhlenbeck Model	8
4.6	Day 6: Log-Likelihood	8
4.7	Day 7: MCMC And Metropolis-Hastings	8
4.8	Day 8: Burn-In And Parameter Extraction	9
4.9	Day 9: Interview Defense	9
5	Code Flow From First Principles	9
5.1	Runtime Flow	9
5.2	Why The Code Is Split Into Modules	10
6	Financial Concepts	10
6.1	Asset	10
6.2	Price Series	10
6.3	Return	10
6.4	Why Log Prices?	10
6.5	Pairs Trading	11
6.6	Hedge Ratio	11
6.7	Correlation Versus Cointegration	11
7	Statistical Foundations	11
7.1	Random Variable	11
7.2	Probability Distribution	11
7.3	Expected Value	11
7.4	Variance	12
7.5	Normal Distribution	12

8	OLS Regression From First Principles	12
8.1	The Model	12
8.2	Objective	12
8.3	Why Squared Errors?	12
8.4	Closed-Form Slope	13
8.5	R-Squared	13
9	Stationarity And ADF	13
9.1	Stationarity	13
9.2	Unit Root	13
9.3	ADF Test	13
9.4	ADF In The Code	14
9.5	Interview Caution	14
10	Stochastic Process Foundations	14
10.1	What Is A Stochastic Process?	14
10.2	Brownian Motion	14
10.3	Stochastic Differential Equation	14
11	Ornstein-Uhlenbeck Process	15
11.1	Definition	15
11.2	Parameter Meaning	15
11.3	Why OU For This Project?	15
11.4	Drift Direction	15
11.5	Exact Discrete Transition	15
11.6	Derivation Sketch	16
12	Likelihood And Log-Likelihood	16
12.1	What Is Likelihood?	16
12.2	Conditional Likelihood	16
12.3	Gaussian Log-Likelihood	17
12.4	Why Use Logs?	17
12.5	Invalid Parameters	17
13	MCMC From First Principles	18
13.1	Why MCMC?	18
13.2	Markov Chain	18
13.3	Monte Carlo	18
13.4	Markov Chain Monte Carlo	18
13.5	Metropolis-Hastings Algorithm	18
13.6	Why Accept Worse Moves?	19
13.7	Proposal Widths	19
13.8	Acceptance Rate	19
14	Burn-In	19
14.1	What Is Burn-In?	19
14.2	Why Median?	20
15	Half-Life	20
15.1	Definition	20
15.2	Interpretation	20
15.3	Trading Interpretation	20

16 Algorithmic Complexity	20
16.1 Data Pipeline	20
16.2 Likelihood	21
16.3 MCMC	21
16.4 Why NumPy Vectorization Helps	21
17 Python And Implementation Details	21
17.1 if <code>__name__ == "__main__"</code>	21
17.2 Pandas DataFrame	21
17.3 NumPy Array	21
17.4 Slicing	22
17.5 Why Preallocate Trace Arrays?	22
18 OOP And Design Questions	22
18.1 Is This Object-Oriented?	22
18.2 Possible Class Design If Needed	22
18.3 Why Not Use Classes Now?	22
19 Limitations And Honest Defenses	23
19.1 GOOG/GOOGL As A Control Pair	23
19.2 AMD/INTC Failure	23
19.3 No Transaction Costs	23
19.4 No Out-Of-Sample Backtest	23
19.5 No Priors	23
20 Common Interview Questions And Strong Answers	24
20.1 Project-Level Questions	24
20.1.1 Q: What is the goal of this project?	24
20.1.2 Q: Why did you choose pairs trading?	24
20.1.3 Q: Why MCMC?	24
20.1.4 Q: Is this a production trading system?	24
20.2 Data Questions	24
20.2.1 Q: Why <code>yfinance</code> ?	24
20.2.2 Q: Why drop NaN rows?	24
20.2.3 Q: Why use adjusted prices?	24
20.3 Regression Questions	24
20.3.1 Q: What is the hedge ratio?	24
20.3.2 Q: What if beta is negative?	24
20.3.3 Q: Does high R-squared mean the pair is tradable?	24
20.4 ADF Questions	25
20.4.1 Q: What is the null hypothesis of ADF?	25
20.4.2 Q: What does p-value below 0.05 mean?	25
20.4.3 Q: Is p-value 0.049 very different from 0.051?	25
20.5 OU Questions	25
20.5.1 Q: Why is OU mean-reverting?	25
20.5.2 Q: What happens if theta increases?	25
20.5.3 Q: What happens if sigma increases?	25
20.5.4 Q: Why is sigma positive?	25
20.6 Likelihood Questions	25
20.6.1 Q: What does log-likelihood measure?	25
20.6.2 Q: Why not use probability directly?	25
20.6.3 Q: Why is the residual squared?	25
20.7 MCMC Questions	26
20.7.1 Q: What is Markov about your chain?	26

20.7.2	Q: What is Monte Carlo about it?	26
20.7.3	Q: Why accept worse proposals?	26
20.7.4	Q: What is acceptance rate?	26
20.7.5	Q: What is a good acceptance rate?	26
20.8	Algorithm Questions	26
20.8.1	Q: What is time complexity?	26
20.8.2	Q: Why preallocate arrays?	26
20.8.3	Q: Why vectorize likelihood?	26
20.9	Software Design Questions	26
20.9.1	Q: Why split into modules?	26
20.9.2	Q: Why not add more abstractions?	26
20.10	Critical Thinking Questions	27
20.10.1	Q: What is the biggest weakness of the project?	27
20.10.2	Q: What would you add next?	27
20.10.3	Q: Why did you not force AMD/INTC to work?	27
21	Line-By-Line Code Explanation	27
21.1	data_pipeline.py	27
21.2	ou_model.py	28
21.3	sampler.py	28
22	Possible Whiteboard Derivations	28
22.1	Derive Half-Life	28
22.2	Derive Metropolis Acceptance Rule	29
23	What To Say If Pressed Hard	29
23.1	Is Your MCMC Actually Bayesian?	29
23.2	Why Use MCMC If OU Has MLE?	29
23.3	What If The Spread Is Not Stationary?	30
23.4	What If Acceptance Rate Is 0.07 Percent?	30
23.5	What If Acceptance Rate Is 90 Percent?	30
24	Defensible Current Results	30
25	Final Interview Checklist	30
26	Short Version To Recite	31
27	Deep-Dive Question Bank	32
27.1	Probability And Statistics	32
27.2	Quant Finance	32
27.3	Stochastic Calculus	33
27.4	Algorithms	33
27.5	Python And Design	33
28	Closing Position	34

1 How To Use This Document

This document is written for an interview where the interviewer may ask questions from quantitative finance, probability, stochastic processes, statistics, algorithms, Python implementation, and software design. The goal is not to make the project sound more advanced than it is. The goal is to know the exact project deeply and to be able to defend every design choice from first principles.

The current project is intentionally simple. It has one main script and three small modules:

File	Purpose
<code>main.py</code>	Runs the whole workflow.
<code>data_pipeline.py</code>	Downloads prices, computes hedge ratio, spread, and ADF test.
<code>ou_model.py</code>	Computes Ornstein-Uhlenbeck log-likelihood.
<code>sampler.py</code>	Runs Metropolis-Hastings MCMC.

If asked what the project does in one sentence:

This project takes two related financial assets, constructs a historical spread between them, checks whether that spread is mean-reverting, and then uses a Metropolis-Hastings MCMC sampler to estimate the hidden parameters of an Ornstein-Uhlenbeck model governing that spread.

2 Project Elevator Pitch

2.1 The Problem

In financial markets, individual stock prices often behave like noisy random walks. Predicting one stock directly is hard because its movement depends on news, macro factors, liquidity, investor behavior, and many unknown shocks.

Pairs trading changes the question. Instead of asking:

Where will stock A go tomorrow?

we ask:

If stock A and stock B usually move together, has their relative price moved too far away from its historical relationship?

The object of interest becomes the **spread**, not the individual prices.

2.2 The Core Idea

Suppose there are two related assets, such as GOOG and GOOGL. If their prices usually move together, then a suitable weighted difference between their log-prices may fluctuate around a stable level. If that spread becomes too high or too low, the model expects it to move back toward its average.

This property is called **mean reversion**.

2.3 What We Estimate

The spread is modeled as an Ornstein-Uhlenbeck process:

$$dX_t = \theta(\mu - X_t) dt + \sigma dW_t.$$

The unknown parameters are:

- θ : speed of mean reversion.
- μ : long-term mean level of the spread.
- σ : volatility/noise level of the spread.

The project estimates these parameters using a Markov Chain Monte Carlo method, specifically the Metropolis-Hastings algorithm.

3 What The Project Is Not

A smart interviewer may test whether you overclaim. Be precise:

- It is not a live trading engine.
- It is not a guaranteed profitable strategy.
- It is not high-frequency trading.
- It does not place real orders.
- It does not include transaction costs, slippage, borrowing cost, or market impact.
- It does not prove the future will behave like the past.
- It is a research prototype for parameter calibration and understanding mean-reverting spreads.

Good interview answer:

The project demonstrates the statistical pipeline: construct a spread, test for stationarity, specify an OU likelihood, and use MCMC to infer parameters. It is not by itself a complete trading system because execution, costs, risk controls, and regime monitoring are not implemented.

4 Day-Wise And Subtask-Wise Plan

4.1 Day 1: Understand The Trading Problem

Goal: Understand why we use pairs rather than single-stock prediction.

Subtasks:

1. Study statistical arbitrage and pairs trading.
2. Understand that the spread is the main object, not the raw prices.
3. Learn why mean reversion matters.
4. Learn the difference between correlation and cointegration.

Interview questions:

- Why not predict stock price directly?
- Why use two assets?
- What does “spread” mean?
- What makes a pair tradable?

Answer: Direct price prediction is difficult because prices are often non-stationary. A spread between related assets may be more stable and mean-reverting. A pairs-trading strategy tries to exploit temporary deviations from the historical relationship.

4.2 Day 2: Data Collection With yfinance

Goal: Download historical price data.

Subtasks:

1. Use `yf.download()` for two tickers.
2. Use adjusted close prices via `auto_adjust=True`.
3. Align dates by dropping missing values.
4. Store prices in a Pandas `DataFrame`.

Why adjusted prices? Adjusted prices account for corporate actions like splits and dividends. Without adjustment, false jumps can enter the data.

Interview trap: If asked whether Yahoo Finance data is production-grade, answer no. It is acceptable for research and student prototypes, but real trading systems require reliable licensed data.

4.3 Day 3: Log Prices And Hedge Ratio

Goal: Estimate how much of asset B should be used to hedge asset A.

Subtasks:

1. Convert raw prices to log prices.
2. Regress $\log(A_t)$ on $\log(B_t)$.
3. Extract intercept α and slope β .
4. Interpret β as the hedge ratio.

Regression model:

$$\log(A_t) = \alpha + \beta \log(B_t) + \epsilon_t.$$

Project spread definition:

$$X_t = \log(A_t) - \beta \log(B_t).$$

The current code prints the intercept but does not subtract it from the spread. That means the long-term mean μ estimated later absorbs the intercept-like level of the spread.

If asked whether one could also use residual spread:

$$\epsilon_t = \log(A_t) - \alpha - \beta \log(B_t).$$

Answer: yes. That would center the spread more directly around zero. The current project keeps the original notebook formula, where the OU mean μ handles the level.

4.4 Day 4: Stationarity And ADF Test

Goal: Check whether the spread is suitable for mean-reversion modeling.

Subtasks:

1. Run Augmented Dickey-Fuller test on the spread.
2. Read the test statistic and p-value.
3. Use p-value below 0.05 as a practical gate.

4. Reject pairs whose spreads look non-stationary.

Interview answer: A mean-reversion model only makes sense if the spread does not wander away permanently. The ADF test checks evidence against the null hypothesis of a unit root. If the p-value is low, we reject the unit-root null and treat the spread as stationary for this prototype.

4.5 Day 5: Ornstein-Uhlenbeck Model

Goal: Choose a stochastic model for the spread.

Subtasks:

1. Understand OU process.
2. Derive conditional mean.
3. Derive conditional variance.
4. Write log-likelihood.

OU process:

$$dX_t = \theta(\mu - X_t) dt + \sigma dW_t.$$

It has two forces:

- deterministic pull back to μ ,
- random Brownian shock through σdW_t .

4.6 Day 6: Log-Likelihood

Goal: Build a score function that says how plausible parameters are.

Subtasks:

1. For each consecutive pair (X_{t-1}, X_t) , compute predicted mean.
2. Compute predicted variance.
3. Compute residual.
4. Sum Gaussian log-probabilities.

This is implemented in `ou_model.py`.

4.7 Day 7: MCMC And Metropolis-Hastings

Goal: Estimate parameters by sampling possible values.

Subtasks:

1. Start from initial guesses.
2. Propose new parameters by random walk.
3. Score old and new parameters.
4. Accept or reject based on likelihood ratio.
5. Store trace arrays.

4.8 Day 8: Burn-In And Parameter Extraction

Goal: Convert raw MCMC traces into final values.

Subtasks:

1. Discard first 25 percent of samples.
2. Take medians of remaining samples.
3. Report θ , μ , σ .
4. Compute half-life.

4.9 Day 9: Interview Defense

Goal: Know limitations and alternatives.

Subtasks:

1. Explain why AMD/INTC failed.
2. Explain why GOOG/GOOGL is a clean control pair.
3. Explain why this is not guaranteed profit.
4. Explain what you would add later, without claiming it is already built.

5 Code Flow From First Principles

5.1 Runtime Flow

When we run:

```
python main.py
```

the following happens:

1. `main.py` imports NumPy, Matplotlib, `calculate_hedge_ratio`, and `run_mcmc_calibrator`.
2. It sets the tickers:
$$A = \text{GOOG}, \quad B = \text{GOOGL}.$$
3. It calls `calculate_hedge_ratio`.
4. That function downloads historical prices.
5. It computes log-prices.
6. It runs OLS regression.
7. It constructs the spread.
8. It runs the ADF stationarity test.
9. Control returns to `main.py`.
10. `main.py` plots the spread.
11. It converts the spread column to a NumPy array.
12. It calls `run_mcmc_calibrator`.
13. The sampler repeatedly calls `compute_log_likelihood`.

14. It returns arrays for θ , μ , and σ .
15. `main.py` discards burn-in and computes medians.
16. It prints calibrated parameters and half-life.

5.2 Why The Code Is Split Into Modules

This is not object-oriented in the heavy sense. It is simple functional compartmentalization:

- `data_pipeline.py`: data and spread construction.
- `ou_model.py`: mathematical likelihood.
- `sampler.py`: MCMC algorithm.
- `main.py`: orchestration.

If asked why not use classes: The current state does not require persistent complex objects. Functions are enough because each phase takes input, produces output, and does not need internal mutable object state. If the project grew to support many pairs, configurations, result storage, and diagnostics, classes like `PairData`, `OUSampler`, and `CalibrationResult` could become useful.

6 Financial Concepts

6.1 Asset

An asset is a tradable financial instrument. In this project, assets are stocks such as `GOOG` and `GOOGL`.

6.2 Price Series

A price series is a sequence:

$$P_1, P_2, \dots, P_T$$

where P_t is the price at time t .

6.3 Return

A simple return is:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

A log return is:

$$r_t = \log(P_t) - \log(P_{t-1}) = \log\left(\frac{P_t}{P_{t-1}}\right).$$

The project does not directly model returns. It models a log-price spread.

6.4 Why Log Prices?

Log prices are useful because percentage changes become differences:

$$\log(P_t) - \log(P_{t-1}) = \log(P_t/P_{t-1}).$$

If one asset trades near 100 and another near 1000, raw price differences are not scale-comparable. Log transformation makes relationships more proportional.

6.5 Pairs Trading

Pairs trading means taking positions in two assets simultaneously so that the portfolio is less exposed to broad market movement and more exposed to relative mispricing.

Example:

- If spread is too high, asset A may be expensive relative to B.
- Strategy may short A and buy B.
- If spread falls back, the combined position profits.

This project does not implement the trade execution. It calibrates the statistical model that could later inform such a strategy.

6.6 Hedge Ratio

The hedge ratio β says how much of asset B is needed to offset asset A in the spread:

$$X_t = \log(A_t) - \beta \log(B_t).$$

If $\beta \approx 1$, one unit of B roughly offsets one unit of A on log scale.

If $\beta > 1$, movements in B need a larger weight.

If $\beta < 0$, the assets move inversely in the regression relationship.

6.7 Correlation Versus Cointegration

Correlation measures whether two series move together at the same time. It is a short-run co-movement measure.

Cointegration means a linear combination of two non-stationary series is stationary. This is stronger and more relevant to pairs trading.

Interview answer: Two assets can be highly correlated but not cointegrated. They may move in the same direction but still drift apart over time. For mean-reversion trading, stationarity of the spread matters more than simple correlation.

7 Statistical Foundations

7.1 Random Variable

A random variable is a quantity whose value depends on chance. In the project, future spread values are treated as random variables.

7.2 Probability Distribution

A probability distribution describes how likely different values are. For a continuous variable, we often use a density function.

7.3 Expected Value

The expected value is a probability-weighted average:

$$\mathbb{E}[X] = \int x f(x) dx.$$

In discrete form:

$$\mathbb{E}[X] = \sum_i x_i p_i.$$

7.4 Variance

Variance measures average squared deviation from the mean:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

In this project, σ controls the volatility/noise of the spread.

7.5 Normal Distribution

A normal distribution is written:

$$X \sim \mathcal{N}(m, v),$$

where m is mean and v is variance. Its density is:

$$f(x) = \frac{1}{\sqrt{2\pi v}} \exp\left(-\frac{(x - m)^2}{2v}\right).$$

The OU transition uses a normal conditional distribution.

8 OLS Regression From First Principles

8.1 The Model

We model:

$$y_t = \alpha + \beta x_t + \epsilon_t,$$

where:

- $y_t = \log(A_t)$,
- $x_t = \log(B_t)$,
- α is intercept,
- β is slope or hedge ratio,
- ϵ_t is residual error.

8.2 Objective

OLS chooses α and β to minimize squared residuals:

$$\min_{\alpha, \beta} \sum_{t=1}^T (y_t - \alpha - \beta x_t)^2.$$

8.3 Why Squared Errors?

Squaring has useful properties:

- positive and negative errors do not cancel,
- large errors are penalized more,
- the objective is differentiable,
- under Gaussian noise, OLS is maximum likelihood.

8.4 Closed-Form Slope

For simple regression:

$$\hat{\beta} = \frac{\sum_t (x_t - \bar{x})(y_t - \bar{y})}{\sum_t (x_t - \bar{x})^2} = \frac{\text{Cov}(x, y)}{\text{Var}(x)}.$$
$$\hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}.$$

8.5 R-Squared

$$R^2 = 1 - \frac{\sum_t (y_t - \hat{y}_t)^2}{\sum_t (y_t - \bar{y})^2}.$$

It measures how much of the variation in y is explained by the regression.

Important: High R^2 does not guarantee stationarity of the spread. That is why we still need the ADF test.

9 Stationarity And ADF

9.1 Stationarity

A time series is stationary if its statistical behavior does not change over time. Weak stationarity usually means:

- constant mean,
- constant variance,
- autocovariance depends only on lag, not calendar time.

For pairs trading, stationarity means the spread is not drifting away permanently.

9.2 Unit Root

A unit-root process has persistent shocks. A simple random walk is:

$$X_t = X_{t-1} + \epsilon_t.$$

If it goes up today, that shock stays embedded forever. Such a process does not mean-revert.

9.3 ADF Test

The Augmented Dickey-Fuller test checks the null hypothesis:

$$H_0 : \text{series has a unit root}$$

against:

$$H_1 : \text{series is stationary}$$

If p-value < 0.05 , we reject the unit-root null at the 5 percent level.

9.4 ADF In The Code

```
adf_result = adfuller(data["Spread"])
```

The code prints:

- ADF statistic,
- p-value,
- critical values,
- success or warning message.

9.5 Interview Caution

Do not say:

ADF proves this spread will always mean-revert.

Say:

ADF gives statistical evidence against a unit root in the historical sample. It supports using a mean-reversion model, but future regimes may change.

10 Stochastic Process Foundations

10.1 What Is A Stochastic Process?

A stochastic process is a collection of random variables indexed by time:

$$\{X_t : t \geq 0\}.$$

In this project, X_t is the spread at time t .

10.2 Brownian Motion

Brownian motion W_t is a continuous-time random process with:

- $W_0 = 0$,
- independent increments,
- $W_t - W_s \sim \mathcal{N}(0, t - s)$ for $t > s$,
- continuous paths.

The term dW_t in the OU equation represents random market shock.

10.3 Stochastic Differential Equation

An SDE has deterministic and random parts:

$$dX_t = a(X_t, t) dt + b(X_t, t) dW_t.$$

For OU:

$$a(X_t, t) = \theta(\mu - X_t), \quad b(X_t, t) = \sigma.$$

11 Ornstein-Uhlenbeck Process

11.1 Definition

$$dX_t = \theta(\mu - X_t) dt + \sigma dW_t.$$

11.2 Parameter Meaning

Parameter	Meaning
μ	Long-run equilibrium level. The spread is pulled toward this.
θ	Pull strength or speed of mean reversion. Larger θ means faster reversion.
σ	Noise intensity. Larger σ means more random movement around the mean.

11.3 Why OU For This Project?

OU is a natural model for mean-reverting continuous variables. The spread is assumed to behave like a noisy rubber band: when far from μ , the drift pulls it back.

11.4 Drift Direction

The deterministic part is:

$$\theta(\mu - X_t).$$

If $X_t > \mu$, then $\mu - X_t < 0$, so drift is negative and pulls downward.

If $X_t < \mu$, then $\mu - X_t > 0$, so drift is positive and pulls upward.

If $X_t = \mu$, deterministic drift is zero.

11.5 Exact Discrete Transition

The code uses the exact OU transition:

$$X_t | X_{t-1} \sim \mathcal{N}(m_t, v_t),$$

where:

$$m_t = X_{t-1}e^{-\theta\Delta t} + \mu(1 - e^{-\theta\Delta t}),$$

$$v_t = \frac{\sigma^2}{2\theta} \left(1 - e^{-2\theta\Delta t}\right).$$

In code:

```
exp_term = np.exp(-theta * dt)
conditional_mean = x_old * exp_term + mu * (1.0 - exp_term)
conditional_var = (sigma**2 / (2.0 * theta)) * (
    1.0 - np.exp(-2.0 * theta * dt)
)
```

11.6 Derivation Sketch

Rewrite:

$$dX_t = \theta\mu dt - \theta X_t dt + \sigma dW_t.$$

Let:

$$Y_t = X_t - \mu.$$

Then:

$$dY_t = -\theta Y_t dt + \sigma dW_t.$$

Solving this linear SDE:

$$Y_t = Y_s e^{-\theta(t-s)} + \sigma \int_s^t e^{-\theta(t-u)} dW_u.$$

Thus:

$$X_t = \mu + (X_s - \mu)e^{-\theta(t-s)} + \sigma \int_s^t e^{-\theta(t-u)} dW_u.$$

The stochastic integral is normally distributed with mean zero and variance:

$$\sigma^2 \int_s^t e^{-2\theta(t-u)} du = \frac{\sigma^2}{2\theta} \left(1 - e^{-2\theta(t-s)}\right).$$

This gives the transition used in the code.

12 Likelihood And Log-Likelihood

12.1 What Is Likelihood?

Probability asks:

Given parameters, how likely is the data?

Likelihood asks:

Given observed data, how plausible are these parameters?

For fixed data, likelihood is a function of parameters:

$$L(\theta, \mu, \sigma \mid X_1, \dots, X_T).$$

12.2 Conditional Likelihood

Because the OU process is Markov:

$$p(X_1, \dots, X_T) = p(X_1) \prod_{t=2}^T p(X_t \mid X_{t-1}).$$

The code effectively scores:

$$\prod_{t=2}^T p(X_t \mid X_{t-1}; \theta, \mu, \sigma).$$

12.3 Gaussian Log-Likelihood

For:

$$X_t | X_{t-1} \sim \mathcal{N}(m_t, v_t),$$

the density is:

$$p(X_t | X_{t-1}) = \frac{1}{\sqrt{2\pi v_t}} \exp\left(-\frac{(X_t - m_t)^2}{2v_t}\right).$$

Taking log:

$$\log p(X_t | X_{t-1}) = -\frac{1}{2} \log(2\pi v_t) - \frac{(X_t - m_t)^2}{2v_t}.$$

Summing over all transitions:

$$\ell(\theta, \mu, \sigma) = -\frac{n}{2} \log(2\pi v) - \frac{1}{2v} \sum_{t=2}^T (X_t - m_t)^2.$$

This is exactly the expression in the code.

12.4 Why Use Logs?

Raw likelihood multiplies many small probabilities. This can underflow to zero in floating-point arithmetic.

Logs convert products into sums:

$$\log\left(\prod_i p_i\right) = \sum_i \log(p_i).$$

This is more numerically stable.

12.5 Invalid Parameters

The code rejects:

$$\theta \leq 0, \quad \sigma \leq 0.$$

Why?

- $\theta \leq 0$ would mean no positive pull back to mean.
- $\sigma \leq 0$ is invalid because volatility cannot be negative.

Code:

```
if theta <= 0 or sigma <= 0:  
    return -np.inf
```

Returning $-\infty$ makes such proposals impossible to accept under the likelihood comparison.

13 MCMC From First Principles

13.1 Why MCMC?

We want the posterior-like distribution of parameters:

$$p(\theta, \mu, \sigma \mid \text{data}).$$

Instead of only finding one best parameter value, MCMC creates a sequence of parameter samples. After burn-in, those samples represent plausible parameter values according to the data and model.

13.2 Markov Chain

A Markov chain is a sequence:

$$S_0, S_1, S_2, \dots$$

where the next state depends only on the current state:

$$P(S_{t+1} \mid S_t, S_{t-1}, \dots, S_0) = P(S_{t+1} \mid S_t).$$

In this project:

$$S_i = (\theta_i, \mu_i, \sigma_i).$$

The next parameter proposal depends only on the current parameter values.

13.3 Monte Carlo

Monte Carlo means using randomness to approximate a quantity. Here, random sampling approximates the parameter distribution.

13.4 Markov Chain Monte Carlo

MCMC combines both:

- Markov chain: each new sample depends only on current sample.
- Monte Carlo: randomness explores the parameter space.

13.5 Metropolis-Hastings Algorithm

At current state s :

$$s = (\theta, \mu, \sigma).$$

Propose:

$$s' = s + \eta,$$

where η is Gaussian random noise.

Compute:

$$\log r = \ell(s') - \ell(s).$$

Accept if:

$$\log u < \log r, \quad u \sim \text{Uniform}(0, 1).$$

If accepted, move to s' . If rejected, stay at s .

13.6 Why Accept Worse Moves?

If s' has lower likelihood, $\log r < 0$. It may still be accepted with probability:

$$\exp(\ell(s') - \ell(s)).$$

This prevents the chain from getting stuck too early in one region.

13.7 Proposal Widths

Current code:

```
proposal_widths = {"theta": 0.005, "mu": 0.00025, "sigma": 0.00025}
```

These are standard deviations of proposal jumps.

If too large:

- proposals jump into unlikely areas,
- acceptance rate becomes too low,
- chain gets stuck.

If too small:

- proposals are accepted often,
- but movement is slow,
- chain explores poorly.

13.8 Acceptance Rate

$$\text{Acceptance rate} = \frac{\text{accepted proposals}}{\text{total proposals}}.$$

For random-walk Metropolis in multiple dimensions, a commonly cited theoretical target is around 23 percent under idealized assumptions. In practical small projects, a broad functional range such as 15–40 percent can be acceptable if the traces mix and results are stable.

Interview answer: Acceptance rate is a diagnostic, not the final proof of correctness. Trace plots, repeated runs, and sensitivity checks are also important.

14 Burn-In

14.1 What Is Burn-In?

The chain starts from arbitrary initial values:

$$\theta_0 = 1.0, \quad \mu_0 = \text{sample mean}, \quad \sigma_0 = \text{sample std.}$$

Early samples may reflect the starting point more than the target distribution. Burn-in discards these early samples.

Current code:

```
burn_in = int(0.25 * len(trace_theta))
clean_theta = trace_theta[burn_in:]
```

This discards first 25 percent.

14.2 Why Median?

The code uses:

$$\hat{\theta} = \text{median of post-burn-in } \theta \text{ samples.}$$

Median is robust to outliers. Mean could also be used.

15 Half-Life

15.1 Definition

For OU mean reversion, expected deviation decays like:

$$\mathbb{E}[X_t - \mu | X_0] = (X_0 - \mu)e^{-\theta t}.$$

Half-life $t_{1/2}$ is the time when expected deviation is half:

$$e^{-\theta t_{1/2}} = \frac{1}{2}.$$

Taking logs:

$$-\theta t_{1/2} = \log(1/2) = -\log 2.$$

Thus:

$$t_{1/2} = \frac{\log 2}{\theta}.$$

15.2 Interpretation

If half-life is 8.37 days, it does not mean the spread must halve exactly after 8.37 days. It means that under the fitted OU model, the expected deviation from mean decays by half in that time.

15.3 Trading Interpretation

Half-life can be used as a time-based risk check:

- If spread has not reverted after several half-lives, model assumptions may have broken.
- Shorter half-life means faster capital recycling.
- Extremely short half-life may be untradable after costs.
- Extremely long half-life may not be useful for a practical pairs trade.

16 Algorithmic Complexity

16.1 Data Pipeline

Let T be number of time points.

- Downloading data: external network/API cost.
- Log transform: $O(T)$.
- OLS for one regressor: effectively $O(T)$.
- ADF test: depends on lag selection, but roughly linear to low-order polynomial in T for this small use case.

16.2 Likelihood

Each likelihood call uses vectorized operations over all $T - 1$ transitions.

Cost:

$$O(T).$$

16.3 MCMC

Let N be iterations.

Each iteration calls likelihood once, so total cost:

$$O(NT).$$

Memory:

$$O(N)$$

for each trace array. There are three arrays: θ , μ , σ .

16.4 Why NumPy Vectorization Helps

The likelihood avoids a Python loop over all data points. Instead, NumPy applies operations over arrays in optimized C-level loops.

This is why:

```
x_old = data[:-1]
x_new = data[1:]
```

is better than manually iterating through each adjacent pair in Python.

17 Python And Implementation Details

17.1 if __name__ == "__main__"

This block runs only when the file is executed directly:

```
if __name__ == "__main__":
    ...
```

If the file is imported by another file, this block does not run.

17.2 Pandas DataFrame

A DataFrame is a table-like structure with rows and columns. Here, dates are the index and ticker symbols are columns.

17.3 NumPy Array

NumPy arrays store numerical data efficiently and support vectorized operations. The spread is converted using:

```
spread_data = processed_df["Spread"].to_numpy()
```

17.4 Slicing

```
x_old = data[:-1]
x_new = data[1:]
```

If:

$$data = [X_0, X_1, X_2, X_3],$$

then:

$$x_{old} = [X_0, X_1, X_2], \quad x_{new} = [X_1, X_2, X_3].$$

This pairs each day with the next day.

17.5 Why Preallocate Trace Arrays?

The code uses:

```
trace_theta = np.zeros(iterations)
```

Preallocating arrays avoids repeated dynamic resizing.

18 OOP And Design Questions

18.1 Is This Object-Oriented?

Not strongly. It is mainly procedural/functional.

Good answer: I kept the implementation simple because the project has one pipeline and one model. The modules separate responsibilities. If the project expanded, object-oriented design could help represent reusable concepts.

18.2 Possible Class Design If Needed

If asked how to convert it to OOP:

```
class PairData:
    def fetch_prices(self): ...
    def compute_spread(self): ...

class OUModel:
    def log_likelihood(self, theta, mu, sigma): ...

class MCMCSampler:
    def run(self): ...

class CalibrationResult:
    def half_life(self): ...
```

18.3 Why Not Use Classes Now?

Because unnecessary classes would make the project look more complex without adding real value. Functions are enough for:

- one data pipeline,
- one likelihood,
- one sampler,
- one script.

19 Limitations And Honest Defenses

19.1 GOOG/GOOGL As A Control Pair

GOOG and GOOGL are share classes of the same company. This makes them a clean control pair for testing the mathematical engine.

But:

- margins may be tiny,
- institutional competition is strong,
- transaction costs can dominate,
- it is not necessarily profitable for retail execution.

Interview answer: I used GOOG/GOOGL because it gives a clean stationary spread to validate the calibration workflow. It is a mathematical sandbox, not a claim that this pair is the best live trade.

19.2 AMD/INTC Failure

AMD and Intel are in the same broad semiconductor sector, but during recent market regimes their business performance diverged. Same sector does not imply stationary spread.

Lesson: The model must test stationarity; it cannot assume it.

19.3 No Transaction Costs

Transaction costs include:

- brokerage fees,
- bid-ask spread,
- slippage,
- short borrowing costs,
- taxes,
- market impact.

This project does not include them.

19.4 No Out-Of-Sample Backtest

The project calibrates parameters on historical data. A full trading project would split data into training and testing periods.

19.5 No Priors

The current sampler uses likelihood and implicit hard constraints ($\theta > 0, \sigma > 0$). It does not explicitly include Bayesian priors.

If asked whether it is fully Bayesian:

It is MCMC-style parameter sampling using a likelihood and hard parameter constraints. A more complete Bayesian version would multiply likelihood by explicit priors for θ , μ , and σ .

20 Common Interview Questions And Strong Answers

20.1 Project-Level Questions

20.1.1 Q: What is the goal of this project?

The goal is to build a simple MCMC-based calibrator for a mean-reverting pairs-trading spread. It downloads historical prices, estimates a hedge ratio, constructs a spread, checks stationarity, models the spread as an OU process, and uses Metropolis-Hastings to estimate θ , μ , and σ .

20.1.2 Q: Why did you choose pairs trading?

Because individual prices are often non-stationary and hard to predict. A spread between related assets can be more stable. Pairs trading focuses on relative mispricing rather than absolute direction.

20.1.3 Q: Why MCMC?

MCMC gives a sampling-based way to explore plausible parameter values. Instead of only returning one point estimate, it builds a trace of parameter values that are likely under the model.

20.1.4 Q: Is this a production trading system?

No. It is a research/calibration prototype. Production would need robust data, out-of-sample testing, transaction costs, execution logic, monitoring, and risk controls.

20.2 Data Questions

20.2.1 Q: Why yfinance?

It is convenient and free for prototyping. It is not institutional-grade data.

20.2.2 Q: Why drop NaN rows?

Both assets must be aligned on the same dates. If one asset has missing data on a date, the spread cannot be computed reliably for that date.

20.2.3 Q: Why use adjusted prices?

Adjusted prices reduce artificial jumps caused by dividends and splits.

20.3 Regression Questions

20.3.1 Q: What is the hedge ratio?

It is the regression slope β in:

$$\log(A_t) = \alpha + \beta \log(B_t) + \epsilon_t.$$

It tells how strongly asset A moves with asset B and how to weight B in the spread.

20.3.2 Q: What if beta is negative?

Then the assets have an inverse relationship in the regression. The spread formula still works mathematically, but trade interpretation changes.

20.3.3 Q: Does high R-squared mean the pair is tradable?

No. High R-squared means good in-sample linear fit, but the spread may still be non-stationary. We need the ADF test.

20.4 ADF Questions

20.4.1 Q: What is the null hypothesis of ADF?

The null is that the series has a unit root, meaning it is non-stationary.

20.4.2 Q: What does p-value below 0.05 mean?

It means we reject the unit-root null at the 5 percent level and treat the spread as stationary for the purposes of this model.

20.4.3 Q: Is p-value 0.049 very different from 0.051?

Not practically. The 0.05 cutoff is conventional. A good researcher would also inspect plots, test robustness, and check out-of-sample behavior.

20.5 OU Questions

20.5.1 Q: Why is OU mean-reverting?

Because the drift term is $\theta(\mu - X_t)$. If X_t is above μ , drift is negative. If below μ , drift is positive.

20.5.2 Q: What happens if theta increases?

The pull back to mean becomes faster. Half-life decreases because:

$$t_{1/2} = \frac{\log 2}{\theta}.$$

20.5.3 Q: What happens if sigma increases?

The spread becomes noisier. Even with mean reversion, random shocks can push it farther from the mean.

20.5.4 Q: Why is sigma positive?

Volatility is a scale parameter. Negative volatility has no meaning in the normal distribution variance.

20.6 Likelihood Questions

20.6.1 Q: What does log-likelihood measure?

It measures how plausible a parameter set is given the observed spread path under the OU transition model.

20.6.2 Q: Why not use probability directly?

Multiplying many small probabilities can underflow to zero. Logs are numerically stable.

20.6.3 Q: Why is the residual squared?

Because the Gaussian density penalizes squared deviations from conditional mean. Large misses receive larger penalty.

20.7 MCMC Questions

20.7.1 Q: What is Markov about your chain?

The next parameter proposal depends only on the current parameter state, not the full past history.

20.7.2 Q: What is Monte Carlo about it?

It uses random sampling to explore parameter space.

20.7.3 Q: Why accept worse proposals?

To avoid getting stuck in local regions and to correctly sample from the target distribution.

20.7.4 Q: What is acceptance rate?

It is the fraction of proposed parameter moves accepted by the sampler.

20.7.5 Q: What is a good acceptance rate?

There is no universal number, but for random-walk Metropolis in multiple dimensions, around 20–40 percent is often healthy; around 23 percent is a classic theoretical reference under ideal assumptions. The current project's 15.7 percent was functional after tuning, but diagnostics should not rely on that number alone.

20.8 Algorithm Questions

20.8.1 Q: What is time complexity?

If T is data length and N is MCMC iterations, each likelihood is $O(T)$ and the sampler is $O(NT)$.

20.8.2 Q: Why preallocate arrays?

It avoids repeated memory allocation during the loop and stores the trace efficiently.

20.8.3 Q: Why vectorize likelihood?

NumPy performs array operations in optimized low-level code, which is faster than Python loops.

20.9 Software Design Questions

20.9.1 Q: Why split into modules?

To separate responsibilities:

- data pipeline,
- OU model,
- sampler,
- orchestration.

20.9.2 Q: Why not add more abstractions?

The current project is small. Adding classes and frameworks would create unnecessary complexity. The structure is simple but still readable.

20.10 Critical Thinking Questions

20.10.1 Q: What is the biggest weakness of the project?

The biggest weakness is that it calibrates on one historical sample without out-of-sample testing or transaction-cost modeling. It proves the calibration workflow, not profitability.

20.10.2 Q: What would you add next?

I would add out-of-sample validation, rolling-window calibration, transaction costs, and trace diagnostics. I would keep them separate from the current simple calibrator.

20.10.3 Q: Why did you not force AMD/INTC to work?

Because a model should reject bad data. If the spread is non-stationary, forcing MCMC on it gives misleading parameters.

21 Line-By-Line Code Explanation

21.1 data_pipeline.py

```
raw_data = yf.download(
    [ticker_a, ticker_b],
    start=start_date,
    end=end_date,
    auto_adjust=True,
    progress=False,
)
```

This downloads historical adjusted price data.

```
data[ticker_a] = raw_data["Close"][ticker_a]
data[ticker_b] = raw_data["Close"][ticker_b]
data = data.dropna()
```

This extracts synchronized close prices and removes missing rows.

```
log_a = np.log(data[ticker_a])
log_b = np.log(data[ticker_b])
```

This transforms prices into log prices.

```
X = sm.add_constant(log_b)
model = sm.OLS(log_a, X).fit()
```

This fits:

$$\log(A_t) = \alpha + \beta \log(B_t) + \epsilon_t.$$

```
data["Spread"] = log_a - (beta * log_b)
```

This creates the project spread.

```
adf_result = adfuller(data["Spread"])
```

This tests whether the spread has a unit root.

21.2 ou_model.py

```
if theta <= 0 or sigma <= 0:  
    return -np.inf
```

Invalid parameters are rejected immediately.

```
x_old = data[:-1]  
x_new = data[1:]
```

This constructs adjacent time pairs.

```
conditional_mean = x_old * exp_term + mu * (1.0 - exp_term)
```

This computes OU conditional mean.

```
conditional_var = (sigma**2 / (2.0 * theta)) * (  
    1.0 - np.exp(-2.0 * theta * dt)  
)
```

This computes OU conditional variance.

```
residuals = x_new - conditional_mean  
sum_squared_residuals = np.sum(residuals**2)
```

This measures prediction errors.

```
log_likelihood = -0.5 * n * np.log(2.0 * np.pi * conditional_var) - (  
    sum_squared_residuals / (2.0 * conditional_var)  
)
```

This is the Gaussian log-likelihood.

21.3 sampler.py

```
current_theta = 1.0  
current_mu = data.mean()  
current_sigma = data.std()
```

These are starting guesses.

```
proposed_theta = current_theta + np.random.normal(0, proposal_widths["theta"])
```

This proposes a new θ by random walk.

```
log_acceptance_ratio = proposed_log_lik - current_log_lik
```

Because log likelihoods are used, likelihood ratio becomes subtraction.

```
if np.log(np.random.uniform(0, 1)) < log_acceptance_ratio:  
    current_theta = proposed_theta
```

This is the Metropolis-Hastings accept/reject rule.

22 Possible Whiteboard Derivations

22.1 Derive Half-Life

Start from expected OU deviation:

$$\mathbb{E}[X_t - \mu] = (X_0 - \mu)e^{-\theta t}.$$

Set:

$$\mathbb{E}[X_t - \mu] = \frac{1}{2}(X_0 - \mu).$$

Cancel:

$$e^{-\theta t} = \frac{1}{2}.$$

Take log:

$$-\theta t = -\log 2.$$

Therefore:

$$t = \frac{\log 2}{\theta}.$$

22.2 Derive Metropolis Acceptance Rule

For symmetric proposal:

$$q(s' | s) = q(s | s').$$

Acceptance probability:

$$\alpha(s, s') = \min \left(1, \frac{\pi(s')}{\pi(s)} \right).$$

If $\pi(s) \propto L(s)$:

$$\alpha = \min \left(1, \frac{L(s')}{L(s)} \right).$$

Taking logs:

$$\log \frac{L(s')}{L(s)} = \ell(s') - \ell(s).$$

The code accepts if:

$$\log u < \ell(s') - \ell(s).$$

23 What To Say If Pressed Hard

23.1 Is Your MCMC Actually Bayesian?

Strict answer:

The implementation uses an MCMC mechanism over a likelihood with hard parameter constraints. It does not include explicit prior distributions, so it is closer to likelihood-based MCMC exploration than a fully specified Bayesian posterior. To make it fully Bayesian, I would define priors such as log-normal or gamma for θ and σ , and normal for μ , then sample from log-posterior = log-likelihood + log-prior.

23.2 Why Use MCMC If OU Has MLE?

Good answer:

For the simple OU model, one can estimate parameters through maximum likelihood or related closed-form/discrete methods. I used MCMC because the project goal is to understand sampling-based calibration and uncertainty over parameters, not only point estimation. MCMC also generalizes when the likelihood becomes more complex.

23.3 What If The Spread Is Not Stationary?

Then the OU assumption is not appropriate. The calibrator may still output numbers, but they should not be trusted. The correct response is to reject the pair or change the time window.

23.4 What If Acceptance Rate Is 0.07 Percent?

The proposal widths are too large relative to the scale of the target distribution. Most proposed parameters land in low-likelihood or invalid regions, so the chain barely moves. I would shrink proposal widths and inspect trace behavior.

23.5 What If Acceptance Rate Is 90 Percent?

Proposal widths are probably too small. The chain accepts many moves but moves very slowly, producing high autocorrelation.

24 Defensible Current Results

From the notebook context, the clean control run produced approximately:

Quantity	Approximate Value
Pair	GOOG/GOOGL
Hedge ratio β	0.997
Intercept α	0.022
R^2	0.9999
ADF p-value	0.0292
Acceptance rate	15.70 percent
θ	0.0829
μ	0.0224
σ	0.0013
Half-life	8.37 days

Defensible interpretation:

- The pair is structurally very close because both are Alphabet share classes.
- The spread passed the ADF gate in the tested sample.
- The estimated θ implies an expected half-life around 8 trading days.
- This is useful for demonstrating the calibrator, not proof of a profitable real strategy.

25 Final Interview Checklist

Before the interview, be able to explain:

1. Why pairs trading uses spread instead of raw prices.
2. Why log prices are used.
3. What OLS estimates.
4. What hedge ratio means.
5. Why high R^2 is not enough.

6. What stationarity means.
7. What ADF tests.
8. What OU process means.
9. What θ , μ , and σ mean.
10. How the OU transition distribution is computed.
11. How the log-likelihood formula is built.
12. Why logs are used.
13. What MCMC means.
14. What Markov means.
15. What Monte Carlo means.
16. How Metropolis-Hastings accepts/rejects.
17. What proposal widths do.
18. What acceptance rate means.
19. Why burn-in is removed.
20. Why median is used.
21. How half-life is derived.
22. Why half-life is not a guarantee.
23. Why GOOG/GOOGL is a control pair.
24. Why AMD/INTC failed.
25. What the project does not include.
26. How the code is compartmentalized.
27. Why the project avoids unnecessary classes.
28. What you would add next without claiming it is already built.

26 Short Version To Recite

If you have only one minute:

This project builds a simple MCMC calibrator for a pairs-trading spread. First, I download adjusted historical prices for two assets and convert them to log prices. I regress log price of asset A on log price of asset B to estimate the hedge ratio, then construct the spread as $\log(A) - \beta \log(B)$. I run an ADF test to check if this spread is stationary, because mean-reversion modeling only makes sense if the spread does not behave like a random walk. Then I model the spread using an Ornstein-Uhlenbeck process with parameters θ , μ , and σ . The likelihood uses the exact OU conditional normal transition. Since the parameters are unknown, I use a Metropolis-Hastings MCMC sampler: propose new parameters, compute log-likelihood, accept better moves and sometimes worse moves according to the likelihood ratio, and store the trace. After burn-in, I take medians as calibrated estimates. Finally, I compute half-life as $\log(2)/\theta$, which gives the expected timescale of mean reversion. The project is a calibration prototype, not a complete trading system.

27 Deep-Dive Question Bank

27.1 Probability And Statistics

1. What is a random variable?
2. What is a probability distribution?
3. What is a density?
4. What is expectation?
5. What is variance?
6. Why is Gaussian distribution used here?
7. What is likelihood?
8. What is log-likelihood?
9. What is maximum likelihood?
10. What is a p-value?
11. What is the null hypothesis in ADF?
12. What is stationarity?
13. What is unit root?
14. What is autocorrelation?
15. Why can high correlation still fail for pairs trading?

27.2 Quant Finance

1. What is pairs trading?
2. What is statistical arbitrage?
3. What is hedge ratio?
4. What is spread?
5. Why use log spread?
6. What is cointegration?
7. Why transaction costs matter?
8. Why is GOOG/GOOGL clean but not necessarily profitable?
9. Why can sector pairs fail?
10. What is regime shift?

27.3 Stochastic Calculus

1. What is Brownian motion?
2. What is an SDE?
3. What is drift?
4. What is diffusion?
5. Why is OU mean-reverting?
6. Derive the OU conditional mean.
7. Derive the OU conditional variance.
8. Derive half-life.

27.4 Algorithms

1. What is MCMC?
2. What is Markov property?
3. What is Monte Carlo?
4. What is Metropolis-Hastings?
5. Why accept worse moves?
6. What is proposal distribution?
7. What are proposal widths?
8. What is burn-in?
9. What is acceptance rate?
10. What is time complexity?
11. What is memory complexity?

27.5 Python And Design

1. Why use Pandas?
2. Why use NumPy?
3. What does vectorization mean?
4. What does slicing do?
5. Why preallocate arrays?
6. Why split files into modules?
7. Why not use classes?
8. How would you convert this into OOP?
9. How would you test this?
10. How would you make it production-ready?

28 Closing Position

The strongest final position in an interview is:

I understand this as a clean calibration prototype. The value of the project is not that it guarantees profit, but that it connects the full chain from data to statistical validation to stochastic modeling to MCMC parameter estimation to a human-interpretable risk metric. I can explain the math and the code, and I also know what the project deliberately does not yet include.